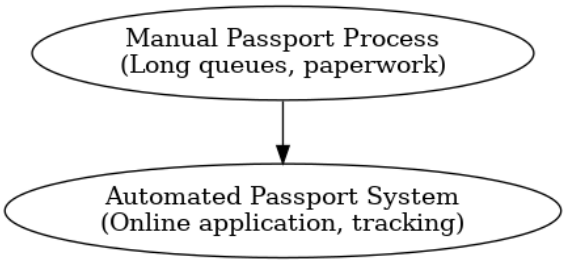
1.Identification of the Problem Statement

a. Passport Automation System

AIM: To identify inefficiencies in the manual passport issuing process and propose automation to ensure speed, transparency, and security.

ProblemStatement : The manual passport issuing process involves filling forms in person, standing in long queues, repeated visits to the office, and delays due to manual verification. The lack of digital tracking makes it hard for applicants to know their status, and paperwork increases the chances of fraud or misplacement.



Challenges in Manual System:

- Long waiting time and repeated visits.
- Human errors in document verification.
- Lack of real-time tracking and transparency.
- Delays due to manual police verification.
- Possibility of fake or tampered documents.

Proposed Automated Solution:

- Online application form with digital document submission.
- Appointment scheduling system to reduce queues.
- Automated police verification request generation.
- Real-time application tracking via portal/SMS.
- Secure digital storage of documents to avoid misplacement.

Benefits: Reduces workload for passport officials.

- Enhances transparency and trust in government services.
- Provides convenience to citizens by allowing anytime, anywhere application.
- Reduces processing time and ensures accuracy.
- Improves data security with digital records.

Scope of the System: Integration with national identity databases.

- Real-time communication between passport office and police departments.
- Multi-language support for inclusivity.

Conclusion: Passport Automation transforms a slow, manual system into a fast, transparent, and secure e-governance solution that benefits both government and citizens.

b. Online Exam Registration

AIM: To identify challenges in the manual exam registration process and propose an online system for accuracy and efficiency.

ProblemStatement:
In traditional exam registration, students must submit physical forms and fees at centers, causing errors in records, queues, and delays in admit card distribution. For large-scale exams (like university or competitive exams), the manual system is unmanageable.

Challenges in Manual System:

- Long queues at registration counters.
- Errors in student details (name, roll number, etc.).
- Delay in admit card generation and distribution.
- Fee mismatches or loss of payment receipts.
- Difficulty in managing thousands of students.

Proposed Automated Solution:

- Online student registration with validation checks.
- Uploading required documents (photo, ID proof, eligibility).
- Secure digital payment gateways for fee collection.
- Automated admit card generation and download facility.
- Real-time center allocation and seat mapping.

Benefits: Saves time for both students and administrators.

- Reduces human errors in data entry.
- Makes fee payment transparent and traceable.
- Easy management of large-scale exams.
- Students get a smooth experience with 24/7 online access.

Scope of the System: Applicable for universities, competitive exams, and certification tests.

- Integration with result management and exam monitoring systems.
- Can be extended for online exams with remote proctoring.

Conclusion: The Online Exam Registration System ensures accuracy, efficiency, and transparency in handling exam processes, providing a better experience for both students and institutions.

C. Book Bank System

AIM: To identify inefficiencies in manual book lending processes and propose a computerized Book Bank management system.

ProblemStatement:
In colleges, Book Banks provide textbooks for students to borrow for a semester. The manual system records details in registers, leading to errors, duplicate issues, and loss of records. Students face inconvenience in knowing book availability, and overdue returns are often not tracked.

Challenges in Manual System:

- Records maintained in registers can be misplaced.
- Duplicate book issues due to lack of tracking.
- Difficult to monitor overdue books and collect penalties.
- Lack of transparency about book availability.
- Staff workload increases with growing student numbers.

Proposed Automated Solution:

- Computerized student enrollment and book catalog management.
- Real-time book availability display for students.
- Barcode/RFID system for quick book issuing and returning.
- Automatic alerts for due dates and penalties.
- Reports for staff to monitor usage and overdue records.

Benefits: Saves time for both staff and students.

- Prevents duplicate or unauthorized book issues.
- Provides transparency by showing available and issued books.
- Helps in efficient penalty collection and reduces financial loss.
- Reduces manual workload and ensures smooth operations.

Scope of the System:

- Can be implemented in schools, colleges, and public libraries.
- Integration with student ID systems for authentication.
- Expansion into digital libraries for e-books.

Conclusion: A computerized Book Bank system solves inefficiencies of manual lending, ensuring transparency, accountability, and convenience for both staff and students.

2. Preparation of Software Requirement Specification (SRS)

a. Passport Automation System

AIM: To prepare a detailed Software Requirement Specification (SRS) that defines the scope, features, and requirements of the Passport Automation System.

Description:

Purpose:
The system aims to simplify and automate the process of passport application, verification, and delivery, making it efficient, transparent, and secure.

Functional Requirements:

- User registration and login with secure credentials.
- Online application form filling with validation checks.
- Uploading necessary documents (ID proof, address proof, etc.).
- Online fee payment using integrated payment gateway.
- Scheduling appointments for physical verification.
- Automated request generation for police verification.
- Tracking passport application status in real-time.
- Admin officer functionalities like verifying documents, updating records, and approving/rejecting applications.

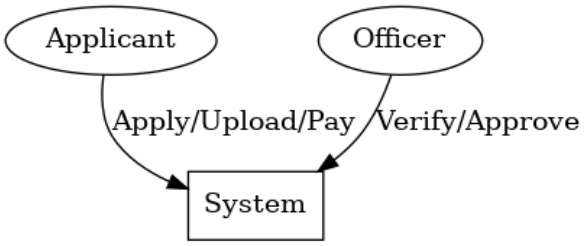
Non-Functional Requirements:

- **Security:** Encrypted storage of sensitive applicant data.
- **Performance:** Ability to handle thousands of concurrent applications.
- **Reliability:** High system uptime (99.9%).
- **Usability:** Easy-to-use interface for applicants and officers.

Constraints: Must comply with government policies and legal frameworks.

- Integration with police database and identity verification systems via secure APIs.
- Support for digital signatures and OTP authentication.

Conclusion: The SRS acts as a blueprint for developers and a reference for stakeholders, ensuring the system fulfills user expectations and adheres to government standards.



b. Online Exam Registration

AIM:
To define detailed software requirements for an Online Exam Registration System that automates the student exam enrollment process.

Description:

Purpose:
The system replaces manual exam registration with a fully digital process, ensuring efficiency, accuracy, and transparency in handling large numbers of students.

Functional Requirements:

- Student registration and login system.
- Exam form filling with personal and academic details.
- Subject selection and exam center preference.
- Uploading documents like photos, signatures, and eligibility certificates.
- Secure online payment gateway for fee submission.
- Automatic admit card generation and download option.
- Notification system via SMS/email for updates.
- Admin functionalities: verifying applications, managing exam centers, and report generation.

Non-Functional Requirements:

- **Security:** Ensure safe transactions and student data protection.
- **Performance:** Handle heavy traffic during peak registration periods.
- **Usability:** User-friendly interface accessible across devices.
- **Reliability:** Ensure uninterrupted access with backup and recovery systems.

Constraints:

- Must follow university/board exam rules.
- Payment gateway compliance with banking standards.
- Storage capacity for thousands of student records.

Assumptions:

- Students have internet access and digital copies of documents.
- Payment gateways function without downtime.

Conclusion:
The SRS ensures smooth communication between developers and exam authorities, avoiding ambiguity and creating a reliable platform for student exam registration.

C. Book Bank System

AIM:
To prepare a Software Requirement Specification document for the automation of Book Bank services in educational institutions.

Description:

Purpose:
The system aims to computerize the manual book lending process, ensuring transparency, efficiency, and proper monitoring of book usage.

Functional Requirements:

- Student registration and login.
- Book catalog with search and filter features.
- Issuing and returning of books with due-date tracking.
- Automatic fine calculation for overdue books.
- Real-time display of available and issued books.
- Librarian/admin functionalities: adding/updating book records, generating reports, monitoring overdue returns.

Non-Functional Requirements:

- **Usability:** Simple and intuitive interface for students and staff.
- **Reliability:** Regular backups of the database to prevent data loss.
- **Performance:** Ability to handle multiple requests simultaneously.
- **Security:** Restrict book issue/return privileges to authorized users only.

Constraints:

- One book per subject rule must be maintained.
- Accurate tracking of available stock in the catalog.
- Integration with student database for authentication.

Assumptions:

- Students have unique IDs and library memberships.
- Library rules (issue limit, fine structure) are predefined.

Stakeholders: Students, Librarians, Administrators

Conclusion: The SRS provides clarity on functional and non-functional needs, guiding developers to build a Book Bank system that reduces errors, increases efficiency, and benefits both students and institutions.

3. Software Configuration Management (SCM) & Risk Management

a. Passport Automation System

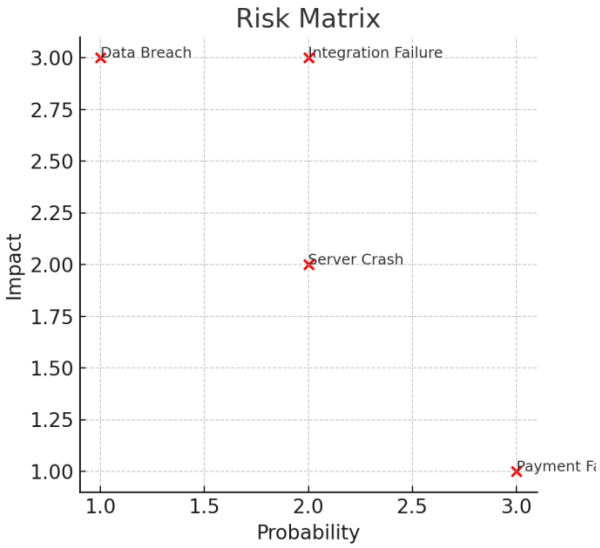
AIM: To manage system versions effectively and handle risks associated with the development, deployment, and maintenance of the Passport Automation System.

Description: Software Configuration Management (SCM):

- SCM ensures proper control of source code, documents, configuration files, and software releases.
- Tools like **Git/GitHub** are used to maintain version history.
- Developers can work on different modules (document upload, fee payment, verification) without overwriting each other's work.
- Branching and merging strategies help in parallel development.
- If an error occurs in the latest version, SCM allows rollback to a stable release.

Risk Management: System downtime during peak passport application periods.

1. **Data security breaches** leading to leakage of sensitive applicant information.
2. **Integration risks** with police verification systems.
3. **Loss of data** due to hardware/software failures.
4. **Mitigation strategies:** Firewalls, intrusion detection systems, and multi-factor authentication.



b. Online Exam Registration

AIM: To apply SCM for controlled system development and address risks for a secure and reliable Online Exam Registration System.

Description:

- SCM organizes and tracks all versions of the system (source code, UI designs, database structures).
- Tools like **Git, SVN** are used for version control.
- Different developers can safely work on **payment module, admit card generation, or login system** simultaneously.
- Any errors introduced in updates can be rolled back using SCM history.
- A configuration baseline ensures stable builds for testing before release.

Risk Management: Major risks:

1. **Server overload** during last-date registrations.
2. **Cyber-attacks** targeting payment gateways.
3. **Data leakage** of student credentials.
4. **Admit card forgery** or misuse.

- **Mitigation measures:**
  - Load balancing and cloud scalability for peak traffic.
  - Secure payment gateways with **SSL/TLS encryption**.
  - CAPTCHAs and OTP authentication for login security.
  - Backup servers for high availability.

C. Book Bank

AIM: To ensure systematic version control and manage operational risks in the computerized Book Bank system.

Description:

- SCM tracks all versions of **book catalog, search modules, issue-return modules, and reports**.
- Tools like Git are used to avoid conflicts when multiple developers update the system.
- Version control ensures proper documentation of changes and rollback in case of failures.
- Stable releases are created and tested before deployment.

Risk Management: Potential risks:

1. **Database corruption** leading to loss of student/book records.
2. **Unauthorized access** to book lending system.
3. **System crashes** during semester book issuing.
4. **Overdue books not tracked properly**, leading to financial loss.

- **Mitigation measures:**
  - Automated daily database backups.
  - Authentication for admin operations.
  - Backup servers for uninterrupted service.
  - Automated fine calculation to manage overdue returns.

Conclusion:
SCM ensures **smooth and conflict-free development**, while risk management ensures the **Book Bank system is secure, stable, and efficient**, especially during peak demand.

4. Usage of CASE Tool

a. Passport Automation System

AIM:
To model the Passport Automation System using Computer-Aided Software Engineering (CASE) tools for effective design and communication.

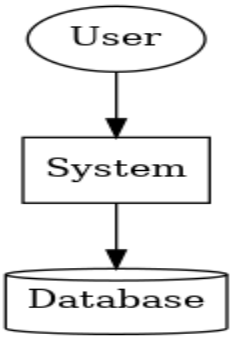
Description:

Introduction to CASE Tools:

- CASE (Computer-Aided Software Engineering) tools such as **StarUML, Lucidchart, and Visual Paradigm** are widely used for creating system design diagrams.
- They help developers visualize the structure, flow, and relationships of a system.

Use Case Diagram:

- Depicts system actors and their interactions.
- Actors: **Applicant, Passport Officer, Admin Officer**.
- Use cases: Registration, Document Upload, Payment, Appointment Scheduling, Police Verification, Passport Issue.
- This clarifies how each user interacts with the system.



Data Flow Diagram (DFD):

- Level 0 DFD shows system overview: User inputs (forms/documents) → Processing (verification, approval) → Outputs (passport issue).
- Level 1/2 DFD further explains modules like Payment Processing or Police Verification.
-

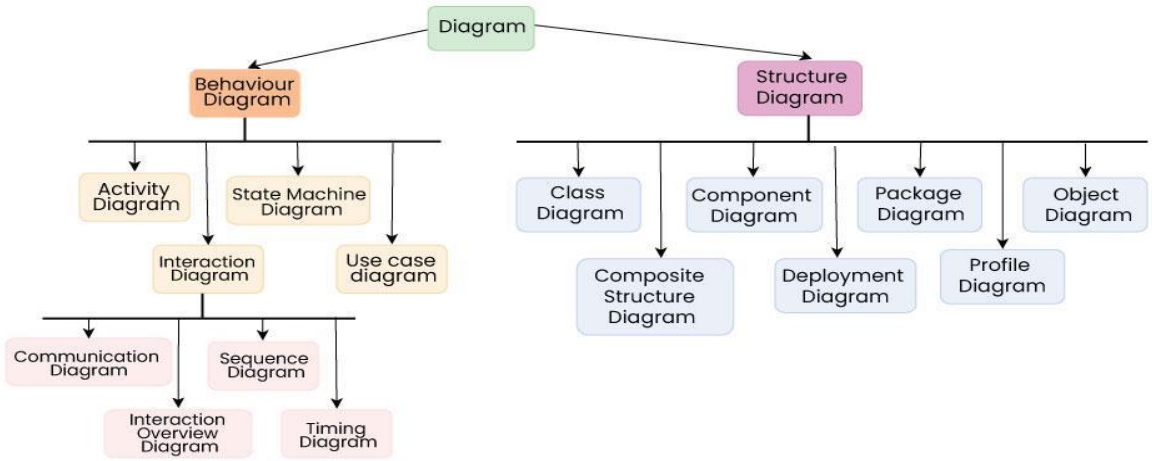
Entity-Relationship Diagram (ERD):

- Identifies **entities**: Applicant, Passport, Appointment, Police Verification, Payment.
- Shows relationships like *Applicant applies for Passport, Passport linked to Police Verification*.
- Ensures database structure is clear.

Advantages:

- Improves stakeholder understanding.
- Detects design flaws early.
- Supports modifications and version control.

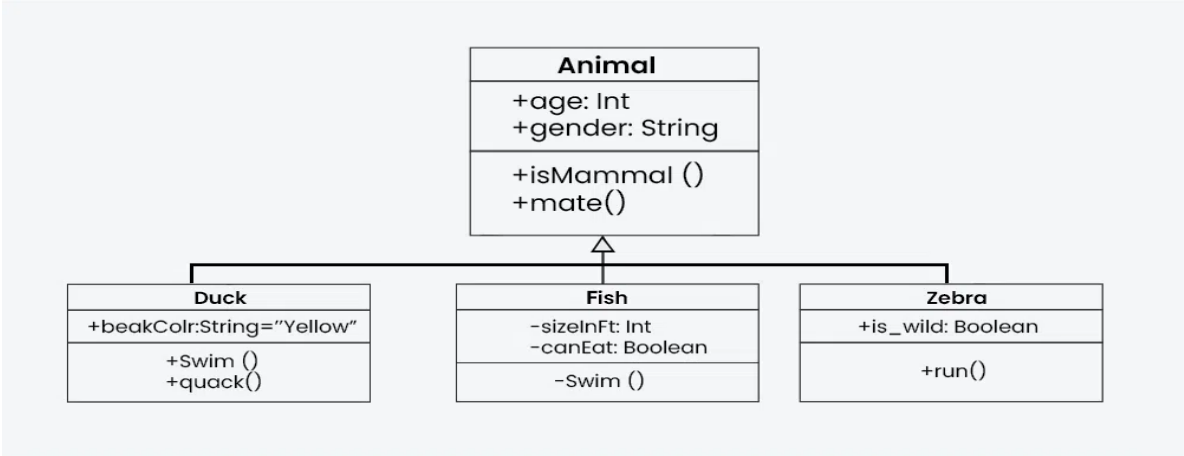
Types of UML Diagrams:



UML Diagrams

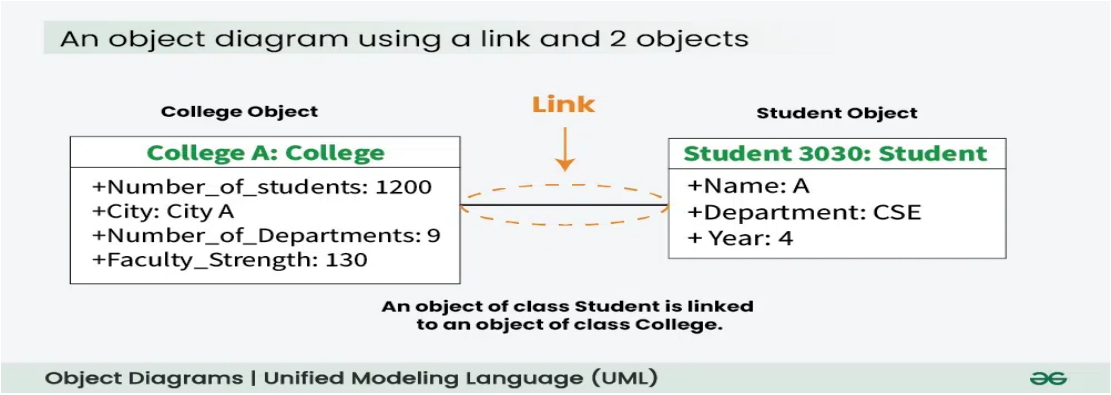
Structural UML Diagrams :

class diagram: It is the building block of all object-oriented software systems.

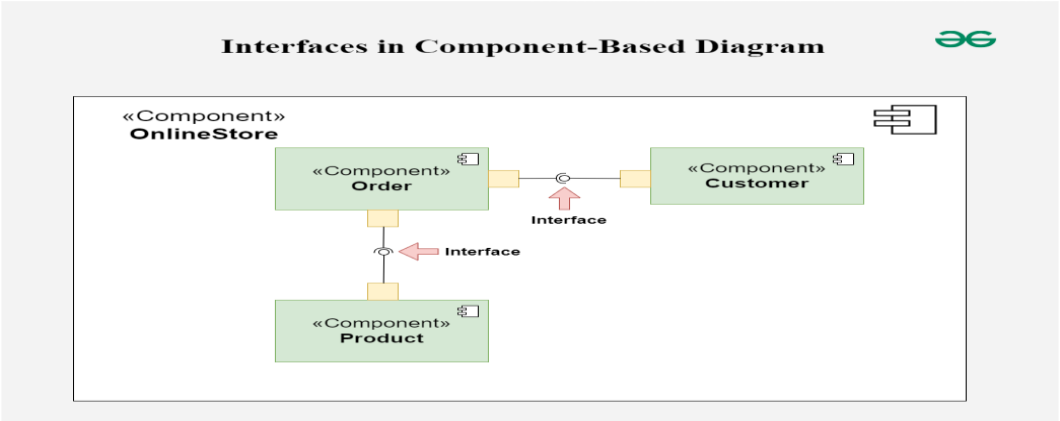


Class Diagram example

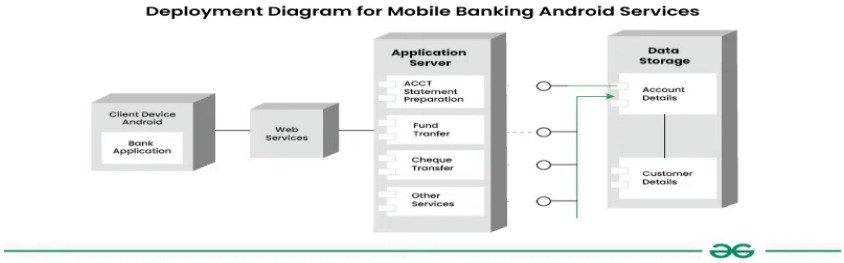
Object Diagram: Can be referred to as a screenshot of the instances in a system and the relationship that exists between them



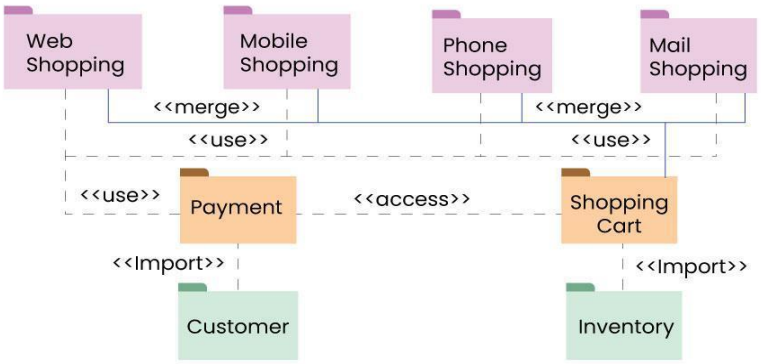
Component diagram: Represent how physical components in system have been organized.system have been organized.



Deployment Diagrams: Represent system hardware and its software.



Package Diagrams: Depict how packages and their elements have been organized.

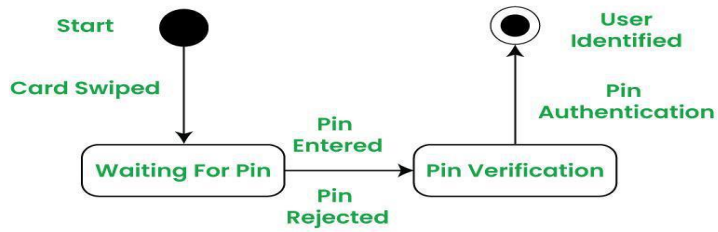


Package Diagram

Behavioral UML Diagrams:

state diagram: Represent the condition or part of the system.

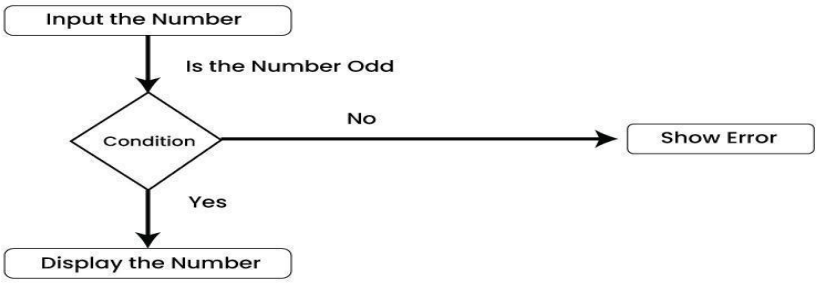
A State Machine Diagram for user verification



State Machine Diagrams | Unified Modeling Language (UML)

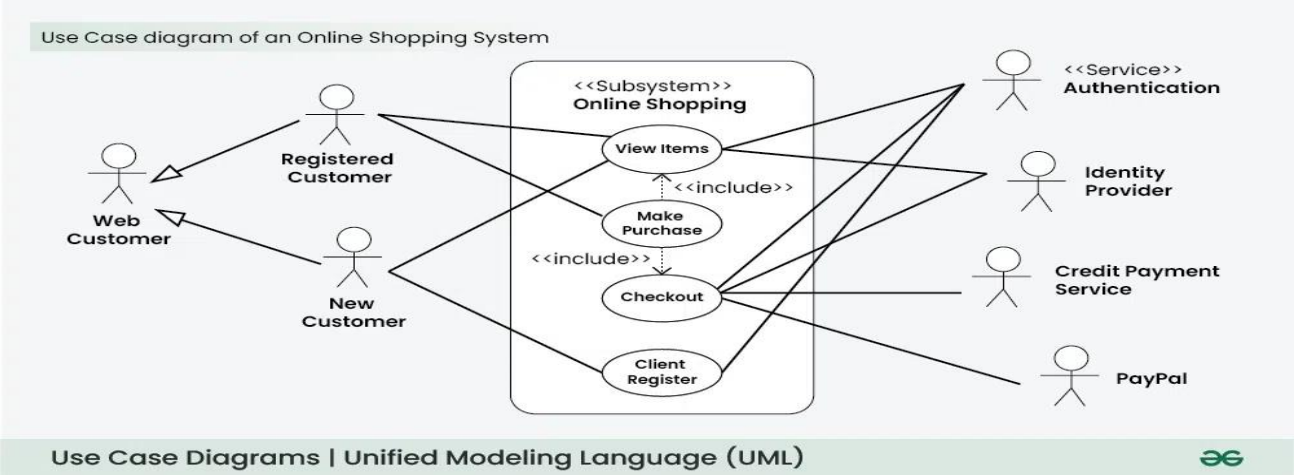
Activity Diagrams: flow of control in a system.

An Activity Diagram using Decision Node

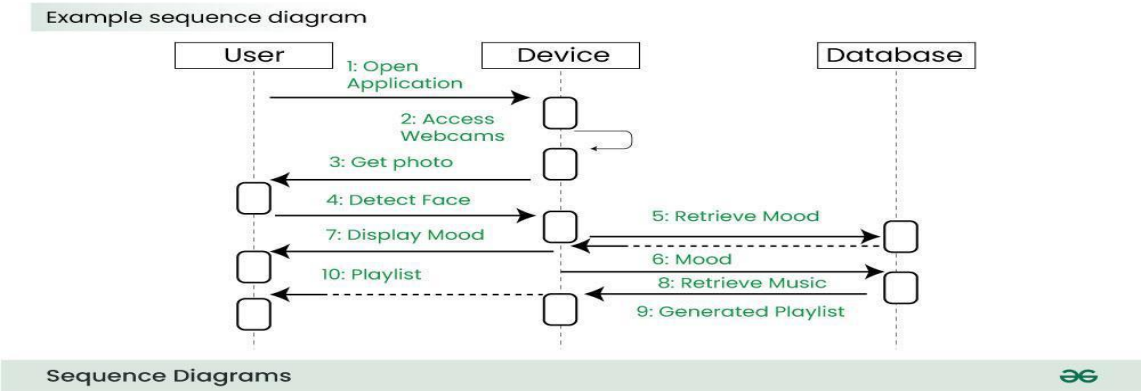


Unified Modeling Language (UML) | Activity Diagrams

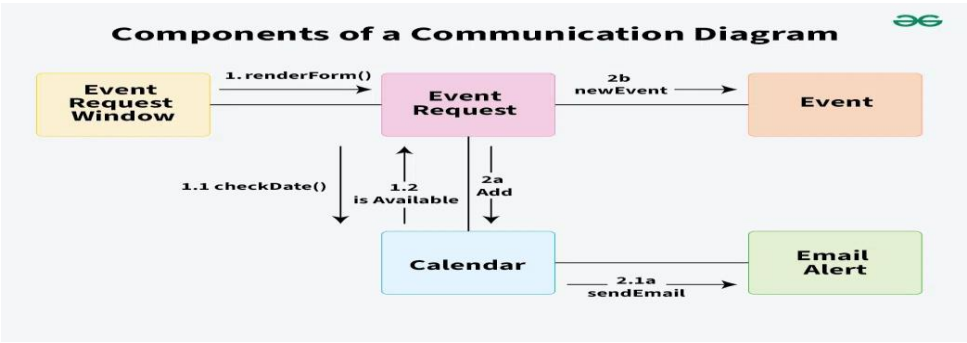
Use Case Diagrams: Depict the functionality or a part of a system.



Sequence diagram: Interaction between objects in a sequential order.



Communication Diagram: Sequenced messages exchanged b/w objects.



b. Online Exam Registration

AIM:
To visualize workflows and requirements of the Online Exam Registration system using CASE tools.

Description:

Introduction to CASE Tools:

- Tools like **Lucidchart, Draw.io, StarUML** help model workflows and entities for the exam registration process.
- They reduce ambiguity and serve as a blueprint for coding.

Use Case Diagram:

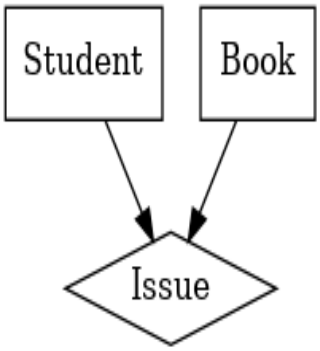
- Actors: **Student, Admin**.
- Student use cases: Registration, Login, Form Filling, Payment, Download Admit Card.
- Admin use cases: Verify Applications, Approve Exams, Generate Reports.

Data Flow Diagram (DFD):

- Level 0: Student enters details → System processes data → Generates Admit Card.
- Level 1: Shows modules like Form Validation, Fee Payment, Notification System.

Entity-Relationship Diagram (ERD):

- Entities: Student, Exam, Payment, AdmitCard.
- Relationships: *Student registers for Exam*, *Payment generates AdmitCard*.
- Ensures proper database schema.



Benefits:

- Simplifies exam workflows.
- Detects missing functionalities before coding.
- Bridges communication between developers, universities, and exam authorities.

Conclusion:
By modeling with CASE tools, the Online Exam Registration system achieves **clarity, consistency, and reduced development errors**.

C. Book Bank

AIM:
To design the Book Bank system visually using CASE tools for clear understanding of processes and relationships.

Description:

Introduction to CASE Tools:

- CASE tools like **StarUML, Lucidchart, Visual Paradigm** help design models for book lending systems.
- These diagrams ensure accurate representation of workflows and database design.

Use Case Diagram:

- Actors: **Student, Librarian, Admin**.
- Student use cases: Search Books, Borrow Books, Return Books.
- Librarian use cases: Verify Student, Update Stock.
- Admin use cases: Generate Reports, Manage Fines.

Data Flow Diagram (DFD):

- Level 0: Student Request → System checks availability → Issue/Return → Update Database.
- Level 1: Breaks into modules like Fine Calculation, Book Catalog Update, Report Generation.

Entity-Relationship Diagram (ERD):

- Entities: Student, Book, IssueRecord, Fine.
- Relationships: *Student borrows Book*, *IssueRecord generates Fine*.
- This ensures accurate database mapping.

Advantages:

- Detects inefficiencies in manual workflows.
- Makes it easier to maintain and expand the system.
- Provides a clear roadmap for development.

Conclusion:
By using CASE tools, the Book Bank system design becomes **transparent, reliable, and efficient**, supporting both developers and administrators.

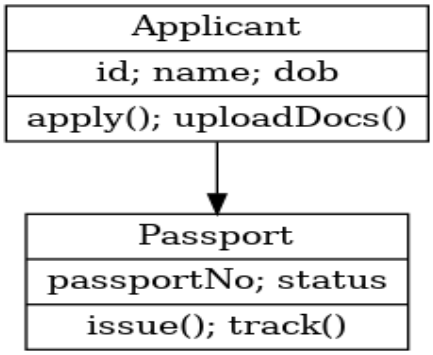
5. Performing the Design using CASE Tools

a. Passport Automation System

AIM: To create detailed system designs for Passport Automation using CASE tools that represent architecture, workflows, and data models.

Description:

- **Class Diagram:**
  - The system is modeled with classes like *Applicant* (ID, name, contact), *Application* (formNo, status, type), *Document* (proofID, passportSizePhoto, addressProof), and *Verification* (policeStatus, adminRemarks).
  - Each class has attributes and operations, e.g., Applicant → apply(), trackStatus(); Verification → approve(), reject().
  - This gives developers clarity on object-oriented structure.



Sequence Diagram:

  - Captures dynamic flow: *Applicant* fills application → *System* validates → *Database* stores data → *Police Module* updates verification → *Passport Officer* issues passport.
  - Each message sequence is visualized step by step.
- **State Diagram:**
  - Models the lifecycle of an application: *Submitted* → *Under Verification* → *Approved* → *Passport Issued* or *Rejected*.
  - Helps in designing transitions clearly.
- **Component Diagram:**
  - Major components: *User Interface, Database, Police Verification Service, Payment Gateway, Admin Dashboard.*
  - Shows how components interact through APIs.
- **Conclusion:**
  These diagrams provide a **complete technical blueprint**, reduce ambiguity, and ensure smooth coding/testing. Developers get both static and dynamic views, making implementation reliable and efficient.

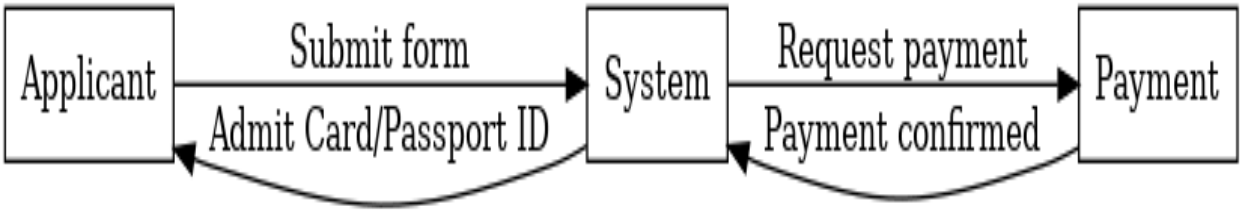
b. Online Exam Registration

AIM: To design the Online Exam Registration system's architecture, structure, and workflows using CASE tools.

Description:

- **Class Diagram:**
  - Key classes: *Student* (id, name, email), *Exam* (examID, subject, center, date), *Payment* (transactionID, amount, status), *AdmitCard* (cardID, studentID, examID).
  - Methods: Student → register(), login(); Payment → process(), verify().

- **Sequence Diagram:**
  - Student submits details → System validates → Payment gateway processes → Confirmation sent → Admit card generated and stored in DB.
  - Shows exact order of interactions.



State Diagram:

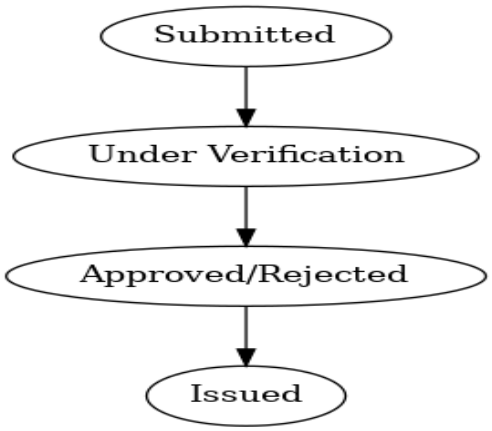
  - Registration lifecycle: *Not Registered → Registered → Payment Pending → Payment Success → Admit Card Generated*.
  - Models how each student's application progresses.
- **Component Diagram:**
  - Main components: *User Interface (website/app), Database, Payment Gateway, Notification System (SMS/Email)*, and *Admin Module*.
  - Highlights communication among subsystems.
- **Conclusion:**
  These design diagrams act as **blueprints for coding and integration**. By modeling the system visually, developers can foresee data flows, error states (like failed payments), and interaction issues, ensuring an error-free exam registration system.

C. Book Bank

AIM:
To design and model the Book Bank system's architecture and workflow using CASE tools.

Description:

- **Class Diagram:**
  - Classes include *Student* (studentID, name, course), *Book* (bookID, title, author, availability), *IssueRecord* (issueID, dateIssued, dueDate, returnStatus), and *Fine* (fineID, amount, daysLate).
  - Librarian and Admin can have specialized methods like addBook(), generateReport().
- **Sequence Diagram:**
  - Flow: Student requests a book → Librarian checks availability → Database updates issue record → Confirmation to student → On return, system updates fine if overdue.
  - Captures real-world sequence.
- **State Diagram:**
  - Book lifecycle: *Available → Issued → Overdue → Returned → Available again*.
  - Ensures fine and availability logic is designed correctly.



Component Diagram:

- Components: *User Interface (student/librarian portals), Database, Report Generator, Notification System (due reminders).*
- Shows how different modules connect for smooth operations.

Conclusion:
By using CASE tools, the Book Bank's design becomes **transparent, well-structured, and reliable**. It ensures that developers build the system aligned with institutional rules, making book lending and fine management efficient.

6. Developing Test Cases for Unit and Integration Testing

a. Passport Automation System

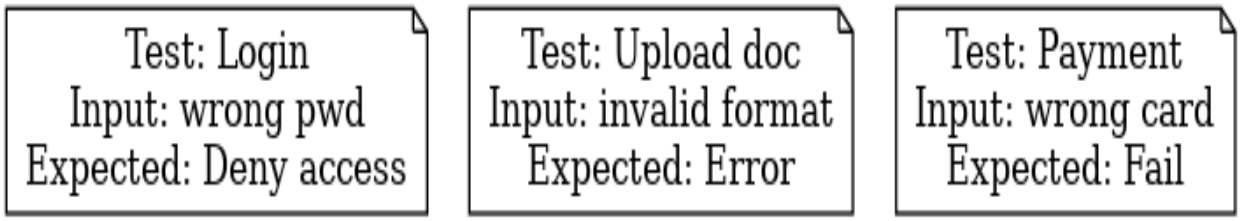
AIM: To verify that each module of the Passport Automation System works correctly in isolation (unit testing) and interacts seamlessly with other modules (integration testing).

Description: Unit testing is performed on the smallest modules of the Passport Automation System to ensure correctness. For example, the **login module** is tested by giving valid and invalid usernames and passwords; expected output is either successful login or error messages.

Similarly, the **document upload module** is tested with PDF, image, and invalid file formats to ensure only acceptable files are uploaded. The **fee payment module** is tested to check accurate calculation of passport fees based on service type (normal, tatkal).

Integration testing validates that all modules work together smoothly. For instance, once an applicant submits the form, the system must correctly send the information to the **police verification module**, and then update the **application status** accordingly. Another integration test is verifying if the **payment gateway** communicates properly with the application and generates a receipt.

Test cases are documented with inputs, expected outputs, and actual results. Example: *Input:* invalid Aadhaar number → *Expected Output:* "Invalid Document" error. *Result:* Pass/Fail. These ensure that bugs are identified early, reducing future risks. Together, unit and integration tests confirm the reliability, security, and performance of the Passport Automation System, making it user-friendly and trustworthy.



b. Online Exam Registration

AIM:
To validate the correctness of modules and verify smooth integration for an error-free Online Exam Registration system.

Description:
Unit testing focuses on individual modules like the **registration form**, which is tested for validating mandatory fields such as name, date of birth, and eligibility criteria. The **subject selection module** is tested with valid and

invalid subject combinations to ensure rules are followed. Another unit test validates the **payment module**, checking if it calculates the fee correctly based on subjects or exam category.

Integration testing ensures modules communicate properly. For example, once a payment is successfully completed, the system must immediately generate an **admit card** and email it to the student. Another integration test verifies if the **exam center selection** passes data accurately to the admit card module. The **payment gateway integration** is tested with both successful and failed transactions to confirm correct system behavior.

Detailed test cases include inputs, expected outputs, and pass/fail conditions. Example: *Input:* invalid card details → *Expected Output:* "Transaction Failed" message. *Result:* Pass. These tests ensure the system is secure, accurate, and reliable during high traffic when thousands of students register simultaneously. By catching bugs early, unit and integration tests guarantee the smooth functioning of the Online Exam Registration system.

C. Book Bank

AIM: To test each Book Bank module individually and confirm their proper integration for reliable library operations.

Description:
Unit testing ensures the correctness of small modules. For example, the **book search module** is tested by searching for existing books (expected → correct results) and non-existent books (expected → "Not Found" message). The **fine calculator module** is tested with different overdue days, ensuring accurate fine amounts are calculated (e.g., 2 days late → ₹10). The **login module** is also tested with valid/invalid student IDs.

Integration testing validates that these modules work together seamlessly. For example, when a student issues a book, the system must automatically update the **book availability** status in the database, link the issue to the **student record**, and generate an **issue receipt**. Another integration test is for book return: the system should update the record, recalculate fines if overdue, and make the book available again.

Test cases are documented carefully: *Input:* Book ID "B102" issued by Student ID "S05" → *Expected Output:* "Book successfully issued, due date 30 days" → *Result:* Pass. This ensures accountability and accuracy.

Unit and integration tests together make the Book Bank system reliable, preventing data loss, duplicate book issues, or incorrect fine calculations. As a result, students and administrators experience a transparent and efficient book lending process.

7. Test Cases for White Box and Black Box Testing

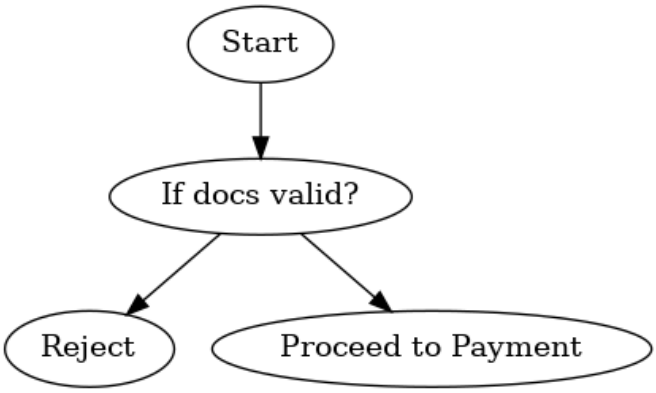
a. Passport Automation System

AIM:
To validate the Passport Automation System using both internal (white box) and external (black box) testing approaches.

Description:
White box testing focuses on examining the internal structure, algorithms, and code of the Passport Automation System. For example, in the **police verification module**, loops that fetch applicant details are tested for efficiency, ensuring they run within expected time limits.**Path coverage** is applied to verify that all logical paths in the code (approved, rejected, pending) are executed at least once. **Condition coverage** ensures that all decision-making branches (if-else conditions) are tested, such as validating document formats or checking payment success.

On the other hand, Black box testing validates the system purely from the user's perspective without knowing the internal code. Functional test cases include entering incomplete details in the application form → expected output: "Error: Please fill all required fields." Another black box test: valid application details → expected output: "Application ID generated successfully." **Boundary testing** checks limits, such as maximum file size upload for documents. **Stress testing** ensures system stability when thousands of users apply simultaneously.

By combining both approaches, developers validate not only the correctness of the code internally but also the external usability and reliability experienced by citizens. This dual testing method guarantees the Passport Automation System is secure, efficient, and user-friendly.



b. Online Exam Registration

AIM:
To test the Online Exam Registration system's internal logic and external functionalities using white box and black box techniques.

Description:
White box testing validates the internal logic of the system. For example, in the **admit card generation module**, a loop ensures every student receives a unique admit card ID; this is tested to confirm no duplication occurs. **Decision coverage** verifies conditions such as "payment success/failure" or "eligibility criteria passed/failed." **Path testing** ensures that all possible execution paths in the code are covered.

Black box testing, meanwhile, focuses on input-output behavior without checking the source code. For instance, if a student enters incorrect login credentials, the system should deny access. Another functional test case: valid subject selection + successful payment → expected output: "Admit Card generated and sent via email." **Boundary value testing** checks the system's handling of maximum allowed subjects per student. **Stress testing** simulates peak traffic, such as thousands of students registering at the same time, to ensure no crashes occur.

Together, white box and black box testing provide complete assurance. While developers validate the internal structure for accuracy, testers validate the system externally for usability and robustness. This ensures that students and exam authorities can rely on the system with confidence.

Input	Expected Output
Valid data	→ Success
Invalid data	→ Error

C. Book Bank

AIM:
To verify both the internal logic and external behavior of the Book Bank system through white box and black box testing methods.

Description:
White box testing is applied to the internal code structure of the Book Bank. For instance, loops that search for books in categories (Science, Literature, Engineering) are tested to ensure they terminate correctly and return accurate results. **Branch coverage** tests all conditions like "book available vs. not available," "student valid vs. invalid." **Path testing** ensures that fine calculation works correctly in all scenarios — returned on time, overdue, or lost book.

Black box testing treats the Book Bank as a black box, focusing only on inputs and outputs. A functional test case: a student attempts to borrow more than the allowed limit of books → expected output: "Request denied." Another case: login with invalid student ID → expected output: "Access denied." **Boundary testing** verifies if the system correctly enforces rules such as "1 book per subject." **Stress testing** checks performance when hundreds of students try to issue or return books simultaneously during semester starts.

By combining both techniques, all hidden coding errors and user-facing issues are detected early. White box testing ensures the correctness of algorithms and logic, while black box testing validates usability and robustness. This dual approach guarantees that the Book Bank system is efficient, secure, and reliable for everyday academic use.